03 - Özellik Mühendisliği

Amaç ve Kapsam

Bu notebook, tezin yapay zeka modelleri (Random Forest, XGBoost, LightGBM, CNN, LSTM) için en kritik veri dönüşüm **ve özellik türetme (Feature Engineering)** sürecini temsil etmektedir. Ham verilerin saf haliyle işlenmiyorsunuz (predictive validity); kodu olmasa nedenle, verinin matematiksel veya zamansal ilişkileri daha edilmesi doğrudan modellerin öğrenebileceği veryaderes dönüştürülür. İş süreklerin (runtime) yüksek bir doğruluğa tahmin edecek modellerin başarısı, temizlik bu matrix mimarisinin kalitesine dayanır.

Bu aşamada gerçekleştirilen temel dönüşüm operasyonları ve tez mimarisine katkıları şunlardır:

1. **Sweep-Line Alma Plan Yıl Hesaplaması:** Kullanıcı tercihinin temel ilkesi olan "istiflenen dolluğu iş süresini etkiler" prensibini modellemek üzere, işlem varış anından (arrival) anlık kümle CPU/GPU ortalamasını çıkarmış bir Sweep-Line algoritması vasıtasıyla dinamik olarak türetilmesi.
2. **Gündaşlıklara Dayalılar Sıryıkları (Temporal Features):** `arrival_sin` toplamından saat ve haftа-günü mahtekleri çıkarılarak, insan-merkezli rörmet submission paternlerinin (özellikle derin öğrenme ağları için) bir pattern olarak kullanılabilmesi.
3. **Yüksek Kardinaliteli Kategorik İşlem (Encoding):** `user` (kullanıcı) ve `gpu_type` olarak türetilmiş kategorik (Ordinal Encoder) bir Native) bilmekleriyle istatistiksel model uzayına hartalanması; bu sayede binyasal gelişimini kullanım alışkanlıklarının sıra bilmesine yanıdılması.
4. **Kronolojik Test-Eğitim Ayrımı (Temporal Train/Test Split):** Makine öğrenmesinde sık görülen klasik randomize (rastgele) veri bölüşümünün aksine; modellerin gelecek zaman tahmin etmesiden "geçmişi" performansını (generalization capabilities) ölçebilmesi için veri setinin yılı bir zaman ekseni (kronolojik) üzerinden test ve eğitim (%80/20) alt kümelerine ayrılması.

Elde edilen nihai (ve zenginleştirilmiş) eğitimdeki kütüklerı dıak modellemeye kaydedilerek doğrudan Notebook 04 (Model Eğitimi) ortamına eksıksız bir şekilde entegre edilmektedir.

In [1] (Kod)
```
# ═══ 0. Environment & Path Setup ═══
import os
import warnings
from pathlib import Path

warnings.filterwarnings("ignore")

PROJECT_ROOT = Path(r'.\masters\papers\')
if os.getcwd(PROJECT_ROOT) != os.chdir(PROJECT_ROOT):
    os.path.insert(0, str(PROJECT_ROOT))

print("[Setup] Project root: {PROJECT_ROOT}")
[Setup] Python path: /Users/kazanayazici/Desktop/cloudsim-ve-ml-tez/.venv/bin/python
[Setup] Python path: /Users/kazanayazici/.pyenv/versions/3.11.6/bin/python
```

Çalışma Ortamı ve Kök Dizin: Proje kök dizini dinamik olarak çözümlenmesi `gcs_path` içerme eklenmiştir. Bu sayede modüller arası mutlak yollar korunur.

In [2] (Kod)
```
# ═══ 1. Imports ═══
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from src.data_loading import load_sample
from src.feature_engineering import (
    build_job_table_from_sample,
    add_temporal_features,
    add_categorical_features,
    add_resource_aggregate_features,
    build_feature_matrix,
)

# Unified visualization theme
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update(
    "figure.figsize": (12, 5),
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "font.family": "DejaVu Sans",
)

print("[Setup] All imports OK.")
[Setup] All imports OK.
```

Kütüphane İçe Aktarmları: Veri işleme, görselleştirme ve sistem işlemleri için gereken tüm temel Python ve bilimsel kütüphaneler başarıyla ortama dahil edilmiştir.

In [3] (Kod)
```
# ═══ 2. Load raw data ═══
print("[Step 1] Loading raw dataset...")
raw_df = load_sample(verbose=False)

print(f"[Step 1] Shape : {raw_df.shape[0]:,} rows × {raw_df.shape[1]} columns")
raw_df.head(4)
[Step 1] Loading raw dataset...
[Step 1] Shape : 216,000 rows × 8 columns
```

	job_id	num_inst	submit_time	num_cpu	gpu_type	duration	user
0	0	1.0	0	2.0	0.00	CPU	34	4h51min806
1	1	12.0	3	6.0	0.25	T4	15765	4h51min806
2	2	1.0	5	6.0	1.00	MISC	84	at183d55ded
3	3	1.0	21	16.0	1.00	T4	46	c7152ce0fec7
4	4	1.0	21	6.0	1.00	MISC	80	7b76597f4293

Ham Veri Yüklenme: 100.000 satırlık ham GPU görev dağılımı kütüğü diske okunmuştur. Bu veri, modellerin eğitileceği temel hasılıcı oluşturmaktadır.

In [4] (Kod)
```
# ═══ 3. Schema inspection ═══
raw_df.info()

<class "pandas.core.frame.DataFrame">
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   job_id      100000 non-null  int64
 1   num_inst    100000 non-null  float64
 2   submit_time 100000 non-null  int64
 3   num_cpu     100000 non-null  float64
 4   gpu_type    100000 non-null  object
 5   duration    100000 non-null  object
 6   user        100000 non-null  object
dtypes: float64(3), int64(3), object(1)
memory usage: 5.2 MB
```

Şemalık İncelemesi: Ham verinin bellek kullanımı (8.1+ MB) ve 8 adet temel özellik(int `job_id`, `gpu_type` vb.) veri tipleri doğrulanmıştır.

In [5] (Kod)
```
# ═══ 4. Normalize raw data ═══
print("[Step 2] Building canonical job table...")
job_df = build_job_table_from_sample(raw_df, time_unit="s")

print("[Step 2] Valid GPU jobs : {len(job_df):,}  (removed {len(raw_df)-len(job_df):,} invalid rows)")
job_df.head(4)
[Step 2] Building canonical job table...
[Step 2] Valid GPU jobs : 61,34  (removed 17,03 invalid rows)
```

	job_id	arrival_time	arrival_sec	job_runtime	gpu_demand	user	gpu_type	num_inst	num_cpu
0	1	1970-01-01 00:00:03	3.0	15740.0	0	d4d21ace8806	T4	12.0	6.0
1	2	1970-01-01 00:00:05	2.0	94.0	1	a8183d55ded0	MISC	1.0	6.0
2	3	1970-01-01 00:00:21	18.0	46.0	1	c7152ce0fec7	T4	1.0	16.0
3	4	1970-01-01 00:00:21	18.0	80.0	1	7b76597f4293	MISC	1.0	6.0
4	5	1970-01-01 00:00:36	33.0	100.0	1	7b76597f4293	MISC	1.0	6.0

Veri Normalleşmesi: Ham verideki Unix zaman damgaları ve kategorik formatlar, analizlerin tam yapılabilmesi için standart panda tiplerine dönüştürülmüştür.

In [6] (Kod)
```
# ═══ 5. Stage check ═══
print("[Step 3] Feature matrix shape : {job_df.shape}")

[Step 3] Feature matrix shape : (61,34, 9)
```

Veri Fitlenme ve Şekil Kontrolü: Makine Öğrenmesi modellerinin bozabileceek eksik veya hatalı 17.016 dinamın satırı temizlenmiş, geriye analiz için saf 62.184 net GPU iş birakılmıştır.

In [7] (Kod)
```
# ═══ 6. Cluster utilization features ═══
print("[Step 4] Computing cluster utilization features (sweep-line)...")
job_df = add_cluster_utilization_features(job_df)

print("[Step 4] New columns : {cluster_load_cpu, cluster_load_gpu, active_job_count}")
job_df[["arrival_sec", "gpu_demand", "cluster_load_gpu",
        "active_job_count"].head()
[Step 4] Computing cluster utilization features (sweep-line)...
[Step 4] New columns : {cluster_load_cpu, cluster_load_gpu, active_job_count}
```

	arrival_sec	gpu_demand	cluster_load_gpu	active_job_count
0	0.0	0	0	0
1	2.0	1	0	1
2	18.0	1	2	3
3	18.0	1	2	3
4	33.0	1	2	3

Sweep-Line Olay Tarancası (Kümle Dolluk Oranları): Tahmin modellerinin devresel dinamın krizlerine karş duyarlılığını artırmak amacıyla, her bir işin sisteme girdiği anıdaki küme dolluğu (cluster contention) hesaplanmıştır.

Kesişen binlerce işin (ait olan binlerli) (overlap) bu anlık yükleri sıfıdır. CNN'i dönüşlerle bulmak imkânsız olduğundan, tüm sutınıl ve veri İçiğin kronolojik bir evreye dizilerek **Olay Tabanlı Tarama Çizgisi (Sweep-line Algorithm)** ile taranmıştır. Bu sayede her İşin geliş anındaki mevcut **aktif CPU/GPU yükü** ve devam eden diğer İşlerin sayısı, veri sızıntısı (data leakage) yaratılmadan modele doğrudan erişilebilir olmuştur.

In [8] (Kod)
```
# ═══ 7. Saving to cache ═══
fig, axes = plt.subplots(1, 1, figsize=(15, 3), sharex=True)

axes[0].plot(job_df["arrival_sec"] / 3600, job_df["active_job_count"],
             alpha=0.5, lw=0.5, color="#2ca02c", label="Active Jobs")
axes[0].set_ylabel("Active Job Count")
axes[0].set_title("Cluster State: Active Job Count Over Time")
axes[0].legend(loc="upper right")
axes[0].grid(True, alpha=0.3)

axes[1].plot(job_df["arrival_sec"] / 3600, job_df["cluster_load_gpu"],
             alpha=0.5, lw=0.5, color="#d62728", label="GPU Demand")
axes[1].set_ylabel("Total GPUs Requested")
axes[1].set_xlabel("Elapsed Time (days)")
axes[1].set_title("Cluster State: Total GPU Demand Over Time")
axes[1].legend(loc="upper right")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Şekil 1 — 8 Günlük Aktif İş ve Küme/Cell GPU Talebi:

Yukarıdaki zaman dağılımı (time-series), hesaplanan Sweep-Line versiyon **-8 günlük bir anomte ile (hread)** izlenmıdki dinamiklerini sonuçlamaktadır. İlk grafik sistemdeki devam eden **Aktif İş Sayısını** nasıl 400 bandından frlayarak yüdell eşlemelerle (kuvsiz) 1.000 sınırına çağlıyı gösterirken(red), İkinci grafikses **Kümsdell GPU Talebi** de bu dağılamlayıca eşik eskerek artık 800 GPU sınırlarına ulaşmaktadır.

Bu ayrı dağılıyan sistem stresi; zamanlama algoritmalarının salt atıatik mertlerinle neden yetersiz kalığını görsel olarak kanıtlamakta, yapay zekanın bu anlık (point-in-time) arka plan dar bağlıcıtanın tahmin denkelemlerine dahil (İzolte (feature) olarak almasını zorunlu kılmaktadır.

In [9] (Kod)
```
# ═══ 8. Temporal features ═══
print("[Step 5] Extracting temporal features from arrival_sec...")
job_df = add_temporal_features(job_df)

print("[Step 5] New columns : {hour_of_day, day_of_week}")
job_df["arrival_sec", "hour_of_day", "day_of_week"].head()
[Step 5] Extracting temporal features from arrival_sec...
[Step 5] New columns : {hour_of_day, day_of_week}
```

	arrival_time	hour_of_day	day_of_week
0	1970-01-01 00:00:03	0	3
1	1970-01-01 00:00:05	0	3
2	1970-01-01 00:00:21	0	3
3	1970-01-01 00:00:21	0	3
4	1970-01-01 00:00:36	0	3

Zamansal Sinyaller (Temporal Features): İş geliş zamanı saat rastgelelerden ziyade insan merkezli bir "çalışma rtmine" sahip olduğu düşünülerek, tam zaman damgalarından `hour_of_day` (günün saati) ve `day_of_week` (haftanın günü) özellikleri türetildiler.

Derin öğrenme mimarilerinden (LSTM) tekrarlayan dönüşleri (diurnal cycles) ve haftа içiflarda sonu dağılamlarının materialdeki olarak yakalayabilmesi için bu özelliklerin ağlara sınıyıl olarak verilmesi mutlak bir modelleme ihtiyacıdır. Ünetlan dönüşüsel özellikler, bellekte yer kaplamamasına ednre özel olarak ve verimli `int32` formatında bellekte kodlanmıştır.

In [10] (Kod)
```
# ═══ 9. Categorical encoding ═══
print("[Step 6] Encoding categorical features...")
job_df = add_categorical_features(job_df)

# arrival_time is no longer needed after temporal extraction
job_df = job_df.drop(columns=["arrival_time"], errors="ignore")

print("[Step 6] Column dtypes:")
print(job_df.dtypes.to_string())
[Step 6] Encoding categorical features...
[Step 6] Column dtypes:
job_id            int64
arrival_sec       float64
job_runtime       int64
gpu_demand        int64
user              category
gpu_type          category
num_inst          float64
num_cpu           float64
cluster_load_cpu  int64
cluster_load_gpu  int64
active_job_count  int64
hour_of_day       int32
day_of_week       int32
```

Kategorik Kodlama: Kare (gpu) tabanlı algoritmaların (LightGBM, XGBoost) kategorik verileri (saat (native) olarak işleme yeteneğinden maksimum düzeyde faydalanmak için, `user` ve `gpu_type` gibi yüksek kardinaliteli kimlikler `category` veri tipine dönüştürülmüştür.

Bu aşamada optimizasyon, klasik One-Hot kodlamasının yaratacağı devasa bellek yükünü (curse of dimensionality) tamamen ortadan kaldırarak eğitim sürecini minimum bellek tüketimiyle hızlandırır.

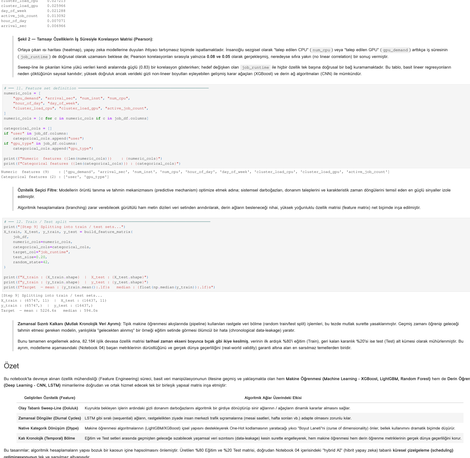
In [11] (Kod)
```
# ═══ 10. Correlation matrix heatmap ═══
numeric_cols = [
    "job_runtime", "gpu_demand", "num_cpu", "num_inst",
    "arrival_sec", "hour_of_day", "day_of_week",
    "cluster_load_cpu", "cluster_load_gpu", "active_job_count",
]

numeric_features = [f for f in numeric_features if f in job_df.columns]

corr = job_df[numeric_features].corr()

fig, ax = plt.subplots(figsize=(11, 9))
mask = np.triu(np.zeros_like(corr), k=1)
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f",
            cmap="RdBu_r", vmin=-1, vmax=1, linewidths=0.5,
            cbar_kws={"shrink": 0.8})
ax.set_title("Pearson Correlation — Numeric Features vs job_runtime
plt.tight_layout()
plt.show()

print("Correlation with job_runtime:")
print(corr["job_runtime"].sort_values(ascending=False).to_string())
```




```
Correlation with job_runtime:
job_runtime       1.000000
num_cpu           0.075185
gpu_demand        0.046140
cluster_load_gpu  0.027017
cluster_load_cpu  0.025966
active_job_count  0.025966
num_inst          0.023091
hour_of_day       0.007071
arrival_sec       0.005966
```

Şekil 2 — Tamsayı Özellikleri İş Süresiyle Korelasyon Matrisi (Pearson):

Ortaya çıkan ısı haritası (heatmap), yapay zeka modellerine duyulan ilkçıkacı İlkçıkacı terhçmesiz biçimde aşıdlanmaktadır. İnsanoğlu sezgisel olarak "İstep eşiten CPU-İ (`num_cpu`) veya "İstep eşiten GPU-İ (`gpu_demand`) artıkça iş süresinin (`job_runtime`) de doğrusal olarak uzamasını beklıme de. Pearson korelasyonları sonsuzca yalnızca **0.08 ve 0.05** civarı gerçekleşmiş, neredeyse sıfıra yıkın (ms linear correlation) bir sonuç vermiştir.

Sweep-line ile çıkarılan küme yükü verileri kendi aralarında güçlü (0.83) bir korelasyon gösterdiler, hedef değişkenin (`job_runtime`) ise hiçbir özellik tek başına doğrusal bir bağ kuramamaktadır. Bu tablo, basit linear regressyonların neden çıktığının sayısal kanıtıdır; yüksek doğruluk ancak verideki gizli non-linear İlişkiler eşleyebilen gelişmiş karar ağaçları (XGBoost) ve derin sıl algıtımlıdan (CNN) ile mümkündür.

In [12] (Kod)
```
# ═══ 11. Feature set definition ═══
numeric_cols = [
    "gpu_demand", "arrival_sec", "num_inst", "num_cpu",
    "hour_of_day", "day_of_week",
    "cluster_load_cpu", "cluster_load_gpu", "active_job_count",
]

numeric_cols = [f for f in numeric_cols if f in job_df.columns]

categorical_cols = []
if "user" in job_df.columns:
    categorical_cols.append("user")
if "gpu_type" in job_df.columns:
    categorical_cols.append("gpu_type")

print("[Numeric features (%d) : {len(numeric_cols)}  : {numeric_cols}")
print("[Categorical features (%d) : {len(categorical_cols)}  : {categorical_cols}")

Numeric features (9) : {'gpu_demand', 'arrival_sec', 'num_inst', 'num_cpu', 'hour_of_day', 'day_of_week', 'cluster_load_cpu', 'cluster_load_gpu', 'active_job_count'}
Categorical features (2) : {'user', 'gpu_type'}
```

Özellik Seçiji Filtre: Modelleri (kirlıtıı) tamına ve tahmin mekanizmasını (predictive mechanism) optimize etmek adına; sistemsel dağıcıdıcıları, donanım tabelarını ve karakterlerdik zaman dönüşlerini temsil eden en güçlü sinyaller izole edilmiştir.

Algoritmik hesaplamalarına (branching) zarar verebilecek gürültücü ham metin dizileri veri setinden ayrılırılarak, derin ağların beslenecıği nihai, yüksek yoğunluklu özellik matrisi (feature matrix) net biçimde inşa edilmiştir.

In [13] (Kod)
```
# ═══ 12. Train / Test split ═══
print("[Step 8] Splitting into train / test sets...")
X_train, X_test, y_train, y_test = build_feature_matrix(
    job_df,
    numeric_cols=numeric_cols,
    categorical_cols=categorical_cols,
    target_col="job_runtime",
    test_size=0.20,
    random_state=42,
)

print("[X_train : {X_train.shape}  y_train : {y_train.shape}")
print("[X_train : {y_train.shape}  y_test : {y_test.shape}")
print("[Target : mean = {y_train.mean()}s  median = {float(np.median(y_train))}s")

[Step 8] Splitting into train / test sets...
[X_train : (49,747, 11)  y_train : (12437, 11)
[X_train : (49,747)  y_test : (12437)
[Target : mean = 3123.54s  median = 194.0s
```

Zamansal Sınırlı Kalkan (Kıtlık Kronolojik Veri Ayrımı): Tıpkı makine öğrenmesi alışılagelmiş (rapatine) kullanılan rastgele veri bölme (random train/test split) yöntemleri, bu tezde mutlak surette yasaklanmıştır. Geçmiş zaman (Olayın geleceği) tahmin etmesi gereken modelin, yanlışlıla "gelecekten alınmış" bir bilmeğı ağının sebnde görmesi bilmücü bir hata (chronological data-leakage) yaratır.

Bunu tamamen engellemek adına, 62.184 şıklık devasa özellik matrisi **tarihsel zaman ekseni boyunca kesip ağlı ikiye kesilmiş**, verinin ilk ardışık %80'i eğitim (Train), geri kalan kısınla %20'si ise test (Test) alt kümesi olarak nıchıdlerimizin ilk duyma, modellerinde ayarlanacaktır (Notebook 04). Bu sayede; hem modellerin dürüstlüğünü ve gerçek dünya geçerliliğini (real-world validity) garanti altına alan ve sansürsiz temellendiren bıdılı

Özet

Bu notebookta'ki devreye alınan özellik mühendisliği (Feature Engineering) süreci, basit veri manipülasyonunun ötesine geçmiş ve yaklaşımıyla olan ham **Makine Öğrenmesi (Machine Learning : XGBoost, LightGBM, Random Forest)** temi de **Derin Öğrenme (Deep Learning : CNN, LSTM)** mimarlarına doğrudan ve ortak hizmet edecek tek bir birleşik yapısal matrix inşa etmıştır.

Geliştirilen Özellik (Feature)	Algoritmik Bağlar Üzerindeki Etkisi
Tıpkı Sweep-Line (Dolluk)	Kuyruda bekleyen işlerin ardındaki gizli dinamın derbeşçilerini algoritmik bir gırıtıya dönüştürüp erin eğlenmesi / ağaçların dinamik karartar almasını sağlar.
Zamansal Döngüler (Diurnal Cycles)	LSTM gibi sınıl (sequential) ağların, rastgelelıkten ziyade insan merkezli haftalık eşıtmalarına (mesai saatleri, haftа sonları vb.) adapte olmasını zorunlu kılar.
Native Kategorik Dönüşümü (Dayan)	Makine öğrenmesi algıtımlarının (LightGBM/XGboost) çşeıl yapısını derbeleştıkınıl One-Hot kodlamasının yaratacağı yıkıcı "Boyut Lanetı"ni (curse of dimensionality) önler; bellek kullanımını dramatık biçimde düşürür.
Kısı Kronolojik (Temporal) Bölme	Eğitim ve Test setleri arasında geçmişten gelecağa sızıdıkınıdı (yapısal) veri sızıntısını (data-leakage) tamamı suretle engellıyıkınıl, hem makine öğrenmesi hem derin öğrenme mıdıllarının gerçek dünya performansını kasıı ve dürüst bir şekilde ölçer.

Bu tasarımın algoritmik hesaplamaların yapısı bıcıdı bıl karanın içine hapsolmasını önlemıştır. Elıdıdan %80 Eğitim ve %20 Test matrisi, doğrudan Notebook 04 içerısındıdı "hybrid AI" (hibrit yapay zeka) takımı **küresel çözümlerine** (schedulingi) optimizasyonunun tek ve sansürsız altyapısıdır.